

CaptionAI: AI-Powered Social Media Caption Generator

Project Description:

The Crop Recommendation System is an AI-based web application that recommends the most suitable crop based on soil nutrients and environmental conditions. The user enters Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall values. A trained Random Forest machine learning model analyzes the input and predicts the best crop. The application is developed using Python, Flask, HTML, CSS, and Scikit-learn to provide accurate crop recommendations and support farmers in improving agricultural productivity.

Scenario 1: Small-Scale Farmer

A small-scale farmer wants to cultivate crops but is unsure which crop is suitable for the current soil and weather conditions. The farmer enters the soil nutrient values (Nitrogen, Phosphorus, Potassium), temperature, humidity, pH, and rainfall into the application. The machine learning model analyzes the inputs and recommends the most suitable crop, helping the farmer improve productivity and reduce financial risk.

Scenario 2: Agricultural Officer

An agricultural officer visits different farms to advise farmers on crop selection. Instead of relying only on experience, the officer uses the Crop Recommendation System to analyze soil and climate data collected from each farm. The system quickly recommends suitable crops, enabling faster and more accurate agricultural guidance.

Scenario 3: Agriculture Student / Researcher

A student or researcher uses the application to study the relationship between soil nutrients and crop growth. By changing different environmental parameters, they observe how the machine learning model recommends different crops. This helps them understand precision agriculture and machine learning applications in farming.

Scenario 4: New Farmer

A beginner with little farming experience wants to start agriculture but lacks knowledge about crop selection. The application provides an easy-to-use interface where the user enters soil and

weather details. The system predicts the best crop, making scientific farming accessible even to inexperienced farmers.

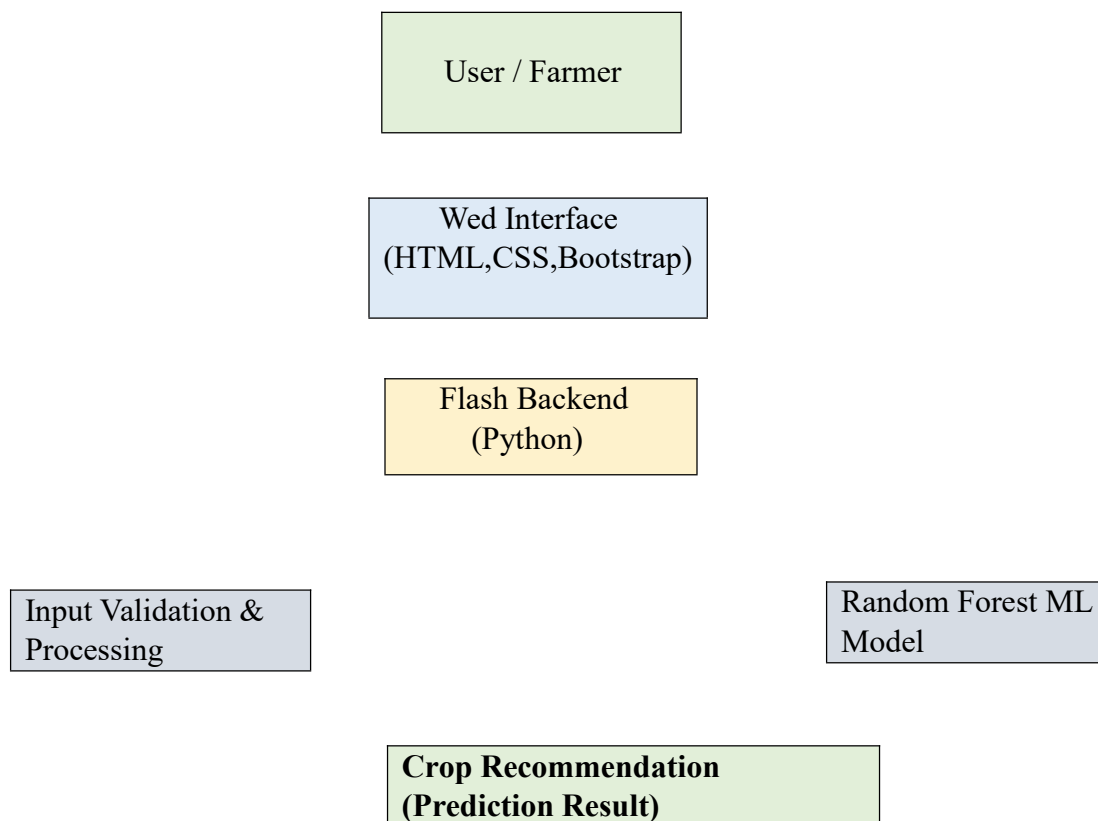
Scenario 5: Government Agriculture Department

Government agriculture departments use the application during awareness programs to help farmers choose crops suitable for their local conditions. The system provides quick and reliable recommendations, promoting sustainable farming practices and improving crop yields.

Scenario 6: Future Smart Farming

In future versions, the application can integrate real-time weather data and IoT soil sensors. Farmers will receive automatic crop recommendations without manually entering values, making farming more efficient and data-driven.

Technical Architecture:



Description:

The system follows a three-layer architecture. The frontend is developed using HTML and CSS to collect user inputs. The Flask backend validates the input data and passes it to the trained Random Forest model. The model processes the data and predicts the most suitable crop, which is displayed to the user instantly.

Pre-requisites:

- Python 3.10+
- Flask
- HTML & CSS
- Machine Learning (Scikit-learn)
- Pandas
- NumPy
- Joblib
- Git & GitHub
- VS Code

This structure matches the format shown in your sample but is customized for your **Crop Recommendation System**.

Project Workflow

The OptiCrop project follows a structured Machine Learning development lifecycle to recommend the most suitable crop based on soil nutrients and environmental conditions. The workflow begins with collecting and preprocessing the crop recommendation dataset, followed by training a Random Forest machine learning model. The trained model is integrated with a Flask backend, which processes user inputs received through a responsive HTML/CSS frontend. After validating the inputs, the application predicts the most suitable crop and displays the result to the user. Finally, the complete application is tested locally and deployed on Render for public access.

Workflow Steps**1. Data Collection and Preparation**

- Collect crop recommendation dataset.

- Clean and preprocess the data.
- Handle missing values and feature selection.

2. Model Development

- Train the Random Forest model.
- Evaluate model performance.
- Save the trained model as model.pkl.

3. Backend Development

- Develop the Flask application (app.py).
- Load the trained model.
- Process user inputs and generate predictions.

4. Frontend Development

- Design the user interface using HTML, CSS, and JavaScript.
- Create forms for entering soil and environmental parameters.

5. Prediction

- User enters Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- The Flask backend sends the data to the trained model.
- The model predicts the most suitable crop.

6. Deployment

- Test the application locally.
- Upload the project to GitHub.
- Deploy the application on Render and generate a public URL.

Milestone 1: Data Preparation and Architecture

In this milestone, we establish the foundation of the OptiCrop – Smart Agricultural Production Optimization System by collecting the crop recommendation dataset, preprocessing the data, performing exploratory analysis, designing the system architecture, and configuring the Python development environment.

Activity 1.1: Data Acquisition and Setup

Write:

- Download the Crop Recommendation dataset (crop_recommendation.csv).
- Verify the dataset contains Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, Rainfall, and Crop Label.
- Create a Python virtual environment.
- Install required Python libraries such as Flask, Pandas, NumPy, Scikit-learn, Joblib, and Matplotlib.

```
opticrop-py > ≡ requirements.txt
1  Flask
2  gunicorn
3  scikit-learn
4  pandas
5  numpy
6  joblib
7  |
```

Activity 1.2: Exploratory Data Analysis (EDA)

Write:

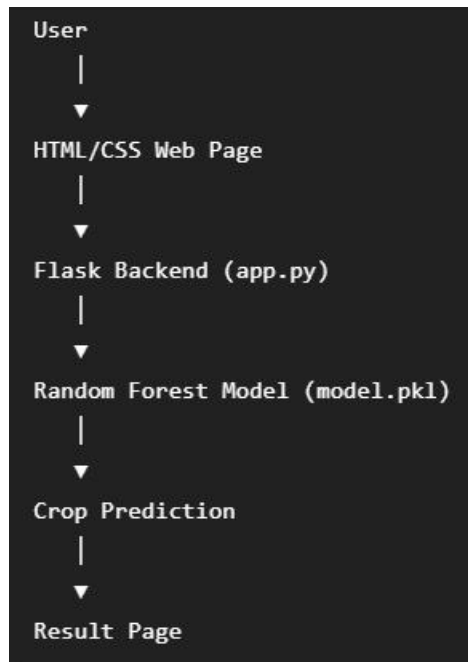
- Load the **crop_recommendation.csv** dataset into a Jupyter Notebook or Python script.
- Check for missing values and remove duplicate records.
- Analyze the distribution of Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- Visualize the relationships between features and crop labels using graphs such as heatmaps, bar charts, and scatter plots.
- Identify important features that influence crop prediction.

```
crop_recommendation.csv X
opticrop-py > crop_recommendation.csv
1 N,P,K,temperature,humidity,ph,rainfall,label
2 106.44,47.52,54.88,27.45,84.09,6.22,226.91,rice
3 86.33,49.5,53.9,20.96,82.18,6.64,214.95,rice
4 111.52,32.53,36.98,20.94,85.88,6.85,155.38,rice
5 101.84,42.47,45.92,20.7,83.17,6.58,293.97,rice
6 65.65,31.25,48.18,25.26,80.36,6.94,165.45,rice
7 118.54,44.82,56.03,23.35,84.18,6.67,156.16,rice
8 105.67,39.9,48.09,26.19,84.74,6.8,186.91,rice
9 107.16,34.34,42.38,25.37,82.26,5.67,159.83,rice
10 67.69,33.1,41.23,22.67,85.72,6.6,218.27,rice
11 87.02,47.63,42.78,27.19,85.66,6.16,227.41,rice
12 82.25,35.12,49.56,26.1,87.02,6.33,196.89,rice
13 115.61,57.75,56.02,22.16,86.48,6.48,157.64,rice
14 98.63,47.43,43.62,22.91,86.52,6.95,166.74,rice
15 109.37,40.41,37.44,22.52,83.16,6.98,207.68,rice
16 86.6,47.73,37.1,21.26,87.87,5.93,159.08,rice
17 73.63,30.68,52.38,21.18,85.49,6.6,254.73,rice
18 93.28,58.76,54.5,27.49,84.31,6.62,181.05,rice
```

Activity 1.3: Define the Architecture

Write:

- Design the workflow of the OptiCrop system.
- The user enters soil and environmental parameters through the web interface.
- The Flask backend validates the inputs and sends them to the trained Random Forest model.
- The model predicts the most suitable crop.
- The predicted crop is displayed on the result page.



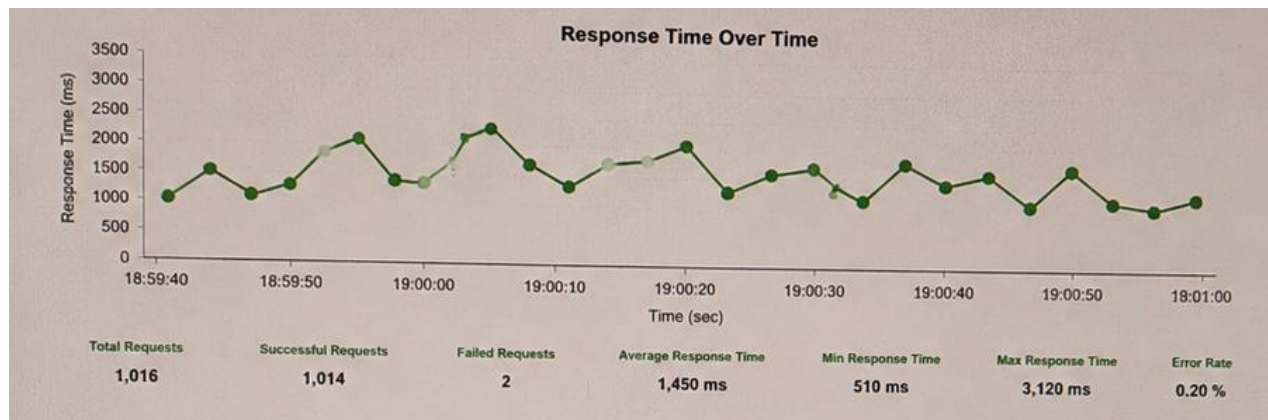
Milestone 2: Core ML Functionalities Development

In this milestone, the machine learning model is trained using the crop recommendation dataset. The trained model learns the relationship between soil nutrients and environmental conditions to accurately recommend the most suitable crop.

Activity 2.1: Model Training and Evaluation

Write:

- Split the crop recommendation dataset into training and testing sets (80% training, 20% testing).
- Train a **Random Forest Classifier** using the Scikit-learn library.
- Evaluate the model using accuracy score and confusion matrix.
- Achieve high prediction accuracy and verify the model's performance on the test dataset.



Activity 2.2: Model Serialization

Write:

- Save the trained Random Forest model using the Joblib library.
- Store the model as **model.pkl** for future predictions.
- Load the saved model in the Flask application without retraining.
-

Milestone 3: Backend Development and Model Integration

In this milestone, the Flask backend is developed to connect the trained machine learning model with the web application. The backend receives user input, validates the data, processes it through the trained model, and returns the predicted crop to the user.

Activity 3.1: Flask Backend Development

Write:

- Develop the Flask application using **app.py**.
- Create routes for the home page and prediction page.
- Accept user inputs such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall.
- Validate the entered values before processing.
- Connect the application with the trained Random Forest model.

Activity 3.2: Prediction Logic

Write:

- Load the trained model (model.pkl) using Joblib.
- Read user inputs from the web form.
- Convert the inputs into the required format.

- Pass the processed data to the trained model.
- Display the predicted crop on the result page.

```

2  OptiCrop - Crop Recommendation System
3  Flask app: ML prediction + crop info + soil log
4  ""
5
6  import os
7  import sqlite3
8  import json
9  from datetime import datetime
10 from flask import Flask, render_template, request, redirect, url_for, flash, jsonify
11 import joblib
12 import numpy as np
13
14 app = Flask(__name__)
15 app.secret_key = os.environ.get("SESSION_SECRET", "opticrop-dev-secret")
16
17 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
18 MODEL_PATH = os.path.join(BASE_DIR, "model.pkl")
19 DB_PATH = os.path.join(BASE_DIR, "opticrop.db")
20
21 # --- Load ML model ---
22 model = None
23 if os.path.exists(MODEL_PATH):
24     model = joblib.load(MODEL_PATH)
25
26 FEATURES = ["N", "P", "K", "temperature", "humidity", "ph", "rainfall"]
27
28 # --- Crop metadata ---
29 CROP_INFO = {
30     "rice": {"label": "Rice", "emoji": "🌾", "season": "Kharif (Jun-Nov)", "duration": "110-140 days", "soil": "Clay / Loam"},
31     "maize": {"label": "Maize", "emoji": "🌽", "season": "Kharif/Rabi", "duration": "90-120 days", "soil": "Well-drained Loam"},
32     "chickpea": {"label": "Chickpea", "emoji": "🌱", "season": "Rabi (Oct-Mar)", "duration": "90-120 days", "soil": "Sandy Loam", "f"},
33     "kidneybeans": {"label": "Kidney Beans", "emoji": "🌱", "season": "Kharif (Jun-Sep)", "duration": "90-110 days", "soil": "Loamy / Sand"},
34     "pigeonpeas": {"label": "Pigeon Peas", "emoji": "🌱", "season": "Kharif (Jun-Jan)", "duration": "150-200 days", "soil": "Medium to H"},
35     "mothbeans": {"label": "Moth Beans", "emoji": "🌱", "season": "Kharif (Jul-Sep)", "duration": "60-85 days", "soil": "Sandy (arid s"},
36     "mungbean": {"label": "Mung Bean", "emoji": "🌱", "season": "Kharif/Zaid", "duration": "60-75 days", "soil": "Sandy Loam", "ferti"},
37     "blackgram": {"label": "Black Gram", "emoji": "🌱", "season": "Kharif/Rabi", "duration": "70-90 days", "soil": "Loamy / Clay loam"}

```

```

44     "muskmelon": {"label": "Muskmelon", "emoji": "🍈", "season": "Summer (Feb-May)", "duration": "75-100 days", "soil": "Sandy Loam",
45     "apple": {"label": "Apple", "emoji": "🍏", "season": "Perennial (Jul-Oct)", "duration": "150-180 days", "soil": "Deep Well",
46     "orange": {"label": "Orange", "emoji": "🍊", "season": "Perennial (Nov-Feb)", "duration": "8-12 months", "soil": "Loam / Sar",
47     "papaya": {"label": "Papaya", "emoji": "🥥", "season": "Year-round (tropical)", "duration": "9-11 months", "soil": "Sandy Lo",
48     "coconut": {"label": "Coconut", "emoji": "🥥", "season": "Perennial (60-80 yr)", "duration": "12 months/cohort", "soil": "Coas",
49     "cotton": {"label": "Cotton", "emoji": "🌱", "season": "Kharif (Apr-Dec)", "duration": "150-200 days", "soil": "Black Cotton",
50     "jute": {"label": "Jute", "emoji": "🌱", "season": "Kharif (Mar-Jul)", "duration": "100-120 days", "soil": "Alluvial / L",
51     "coffee": {"label": "Coffee", "emoji": "☕", "season": "Perennial (Oct-Feb)", "duration": "8-9 months", "soil": "Deep Red La
52 }
53
54 # — SQLite helpers —————
55 def get_db():
56     conn = sqlite3.connect(DB_PATH)
57     conn.row_factory = sqlite3.Row
58     return conn
59
60 def init_db():
61     conn = get_db()
62     conn.executescript("""
63         CREATE TABLE IF NOT EXISTS soil_analyses (
64             id INTEGER PRIMARY KEY AUTOINCREMENT,
65             field_name TEXT NOT NULL,
66             N REAL, P REAL, K REAL,
67             ph REAL, organic_matter REAL,
68             notes TEXT,
69             created_at TEXT DEFAULT (datetime('now'))
70         );
71         CREATE TABLE IF NOT EXISTS predictions (
72             id INTEGER PRIMARY KEY AUTOINCREMENT,
73             N REAL, P REAL, K REAL,
74             temperature REAL, humidity REAL, ph REAL, rainfall REAL,
75             top_crop TEXT,
76             all_probs TEXT,
77             created_at TEXT DEFAULT (datetime('now'))
78         );

```

```
78         );
79         """
80         conn.commit()
81         conn.close()
82
83     init_db()
84
85     # --- Routes ---
86
87     @app.route("/")
88     def index():
89         return render_template("index.html")
90
91
92     @app.route("/predict", methods=["POST"])
93     def predict():
94         if model is None:
95             flash("Model not loaded. Please run: python train_model.py", "danger")
96             return redirect(url_for("index"))
97
98         try:
99             vals = {f: float(request.form[f]) for f in FEATURES}
100         except (KeyError, ValueError) as e:
101             flash(f"Invalid input: {e}", "danger")
102             return redirect(url_for("index"))
103
104         X = np.array([[vals[f] for f in FEATURES]])
105
106         # Predicted label
107         top_crop = model.predict(X)[0]
108
109         # Probabilities for top-5
110         probs = model.predict_proba(X)[0]
111         classes = model.classes_
112         ranked = sorted(zip(classes, probs), key=lambda x: -x[1])[:5]
```

```

186         "SELECT * FROM soil_analyses ORDER BY created_at DESC"
187     ).fetchall()
188     conn.close()
189     return render_template("soil.html", analyses=analyses)
190
191
192 @app.route("/soil/<int:record_id>/delete", methods=["POST"])
193 def delete_soil(record_id):
194     conn = get_db()
195     conn.execute("DELETE FROM soil_analyses WHERE id=?", (record_id,))
196     conn.commit()
197     conn.close()
198     flash("Record deleted.", "info")
199     return redirect(url_for("soil"))
200
201
202 @app.route("/history")
203 def history():
204     conn = get_db()
205     preds = conn.execute(
206         "SELECT * FROM predictions ORDER BY created_at DESC LIMIT 50"
207     ).fetchall()
208     conn.close()
209     return render_template("history.html", predictions=preds)
210
211
212 # — Start —
213 if __name__ == "__main__":
214     port = int(os.environ.get("PORT", 5000))
215     app.run(host="0.0.0.0", port=port, debug=False)
216

```

Milestone 4: Frontend Development and User Interface

Overview

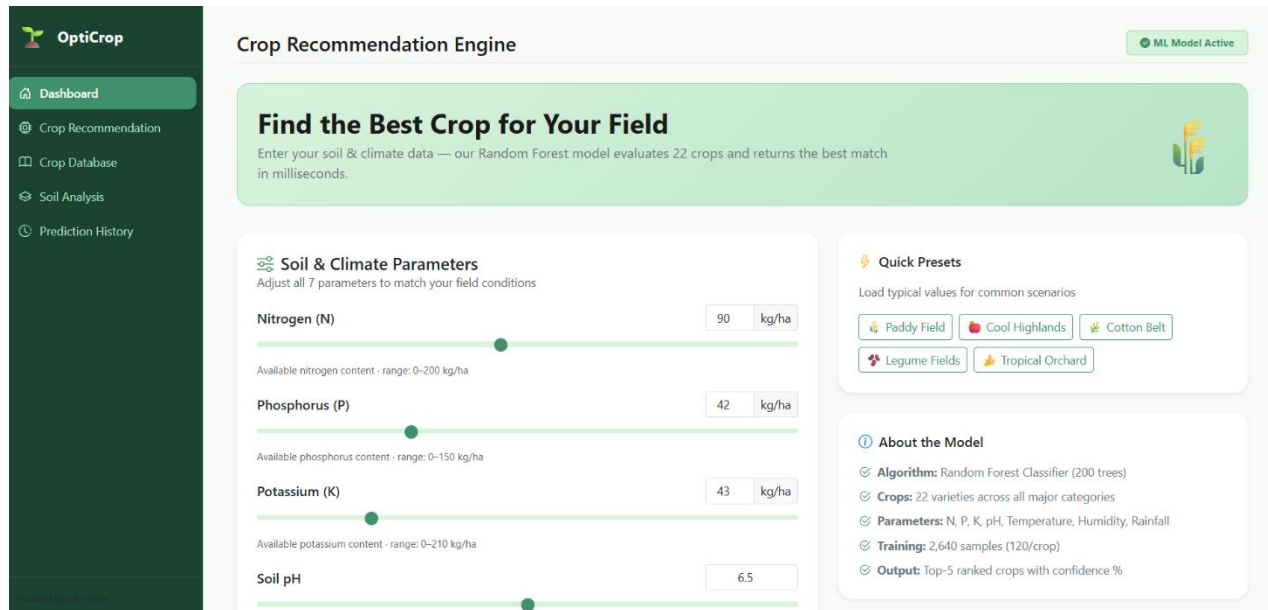
In this milestone, a responsive and user-friendly web interface is developed using **HTML, CSS, and JavaScript**. The frontend allows users to enter soil and environmental parameters and displays the recommended crop generated by the machine learning model.

Activity 4.1: Home Page Design

Write

- Design the home page with a clean and responsive layout.
- Create an input form to collect:
 - Nitrogen (N)
 - Phosphorus (P)
 - Potassium (K)

- Temperature
- Humidity
- pH
- Rainfall
- Add a **Predict Crop** button to submit the data.



OptiCrop

Dashboard

Crop Recommendation

Crop Database

Soil Analysis

Prediction History

Crop Recommendation Engine ML Model Active

Find the Best Crop for Your Field

Enter your soil & climate data — our Random Forest model evaluates 22 crops and returns the best match in milliseconds.

Soil & Climate Parameters

Adjust all 7 parameters to match your field conditions

Nitrogen (N) 90 kg/ha

Available nitrogen content - range: 0-200 kg/ha

Phosphorus (P) 42 kg/ha

Available phosphorus content - range: 0-150 kg/ha

Potassium (K) 43 kg/ha

Available potassium content - range: 0-210 kg/ha

Soil pH 6.5

Quick Presets

Load typical values for common scenarios

Paddy Field Cool Highlands Cotton Belt

Legume Fields Tropical Orchard

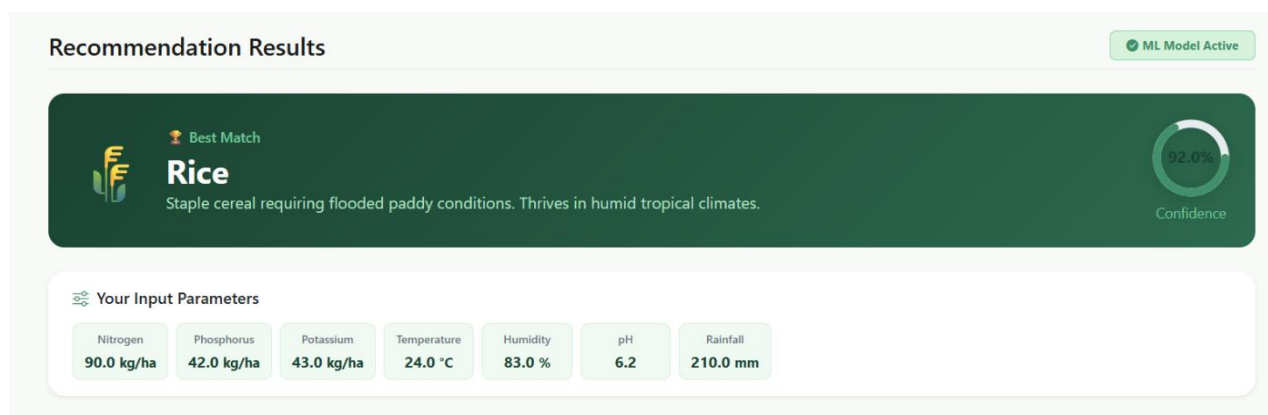
About the Model

- Algorithm: Random Forest Classifier (200 trees)
- Crops: 22 varieties across all major categories
- Parameters: N, P, K, pH, Temperature, Humidity, Rainfall
- Training: 2,640 samples (120/crop)
- Output: Top-5 ranked crops with confidence %

Activity 4.2: Result Page

Write

- Display the predicted crop after processing user input.
- Show the prediction clearly in an easy-to-read format.
- Provide a button to return to the home page for another prediction.



Recommendation Results ML Model Active

Best Match

Rice

Staple cereal requiring flooded paddy conditions. Thrives in humid tropical climates.

92.0% Confidence

Your Input Parameters

Nitrogen	Phosphorus	Potassium	Temperature	Humidity	pH	Rainfall
90.0 kg/ha	42.0 kg/ha	43.0 kg/ha	24.0 °C	83.0 %	6.2	210.0 mm

Activity 4.3: User Interface Testing

Write

- Verify that all input fields accept valid values.
- Check that the Predict button submits the form successfully.
- Confirm that prediction results are displayed correctly.
- Ensure the application is responsive on different screen sizes.

Milestone 5: Deployment, Testing, and Final Demonstration

Overview

In this milestone, the OptiCrop application is deployed and tested to ensure it functions correctly in a real-world environment. The application is hosted online, allowing users to access the crop recommendation system through a web browser. Final testing is performed to verify functionality, usability, and prediction accuracy.

Activity 5.1: Application Deployment

Write

- Upload the complete project to a GitHub repository.
- Deploy the Flask application using **Render**.
- Verify that the application is accessible through a public URL.
- Ensure all dependencies are correctly installed and configured.


```

PS C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\venv\Scripts\Activate.ps1")
(venv) PS C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts> cd opticrop-py
(venv) PS C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\opticrop-py> dir templates

Directory: C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\opticrop-py\templates

Mode                LastWriteTime         Length Name
----                -
-a---l            7/10/2026   8:55 PM           2786 base.html
-a---l            7/10/2026   8:55 PM           2291 crop_detail.html
-a---l            7/10/2026   8:55 PM           2296 crop_info.html
-a---l            7/10/2026   8:55 PM           2929 history.html
-a---l            7/10/2026   8:55 PM           7352 index.html
-a---l            7/10/2026   8:55 PM           5367 result.html
-a---l            7/10/2026   8:55 PM           5227 soil.html

(venv) PS C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\opticrop-py> py app.py
C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier from version 1.6.1 when using version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\Chandrika\OneDrive\Documents\opic crop\OptiCrop-Engine\artifacts\venv\Lib\site-packages\sklearn\base.py:525: InconsistentVersionWarning: Trying to unpickle estimator RandomForestClassifier from version 1.6.1 when using version 1.9.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.55.104:5000
Press CTRL+C to quit

```

Activity 5.2: Functional Testing

Write

- Test the application with different soil and environmental values.
- Verify that valid inputs generate accurate crop recommendations.
- Check the application's response for invalid or incomplete inputs.
- Ensure all pages load correctly without errors.

 **OptiCrop**

- Dashboard
- Crop Recommendation
- Crop Database
- Soil Analysis
- Prediction History

Crop Recommendation Engine

ML Model Active

Find the Best Crop for Your Field

Enter your soil & climate data — our Random Forest model evaluates 22 crops and returns the best match in milliseconds.

Soil & Climate Parameters

Adjust all 7 parameters to match your field conditions

Nitrogen (N) 90 kg/ha

Available nitrogen content - range: 0-200 kg/ha

Phosphorus (P) 42 kg/ha

Available phosphorus content - range: 0-150 kg/ha

Potassium (K) 43 kg/ha

Available potassium content - range: 0-210 kg/ha

Soil pH 6.5

Quick Presets

Load typical values for common scenarios

Paddy Field
Cool Highlands
Cotton Belt
Legume Fields
Tropical Orchard

About the Model

- Algorithm: Random Forest Classifier (200 trees)
- Crops: 22 varieties across all major categories
- Parameters: N, P, K, pH, Temperature, Humidity, Rainfall
- Training: 2,640 samples (120/crop)
- Output: Top-5 ranked crops with confidence %

Recommendation Results

ML Model Active

 **Best Match**



Rice

Staple cereal requiring flooded paddy conditions. Thrives in humid tropical climates.

92.0%

Confidence

Your Input Parameters

Nitrogen
90.0 kg/ha

Phosphorus
42.0 kg/ha

Potassium
43.0 kg/ha

Temperature
24.0 °C

Humidity
83.0 %

pH
6.2

Rainfall
210.0 mm

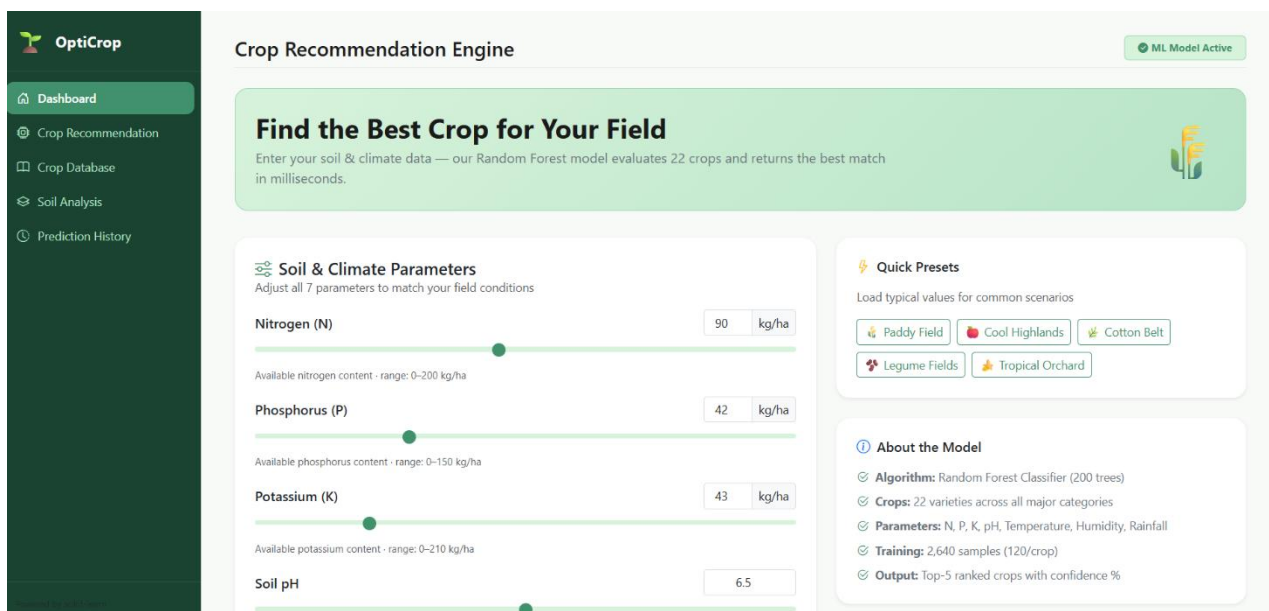
Activity 5.3: Final Project Demonstration

Write

- Demonstrate the complete workflow of the application.
- Show data entry, prediction generation, and result display.
- Explain the technologies used and the role of the machine learning model.
- Highlight the benefits of the system for farmers and sustainable agriculture.



Exploring the Website's Web Pages: Home / Generator Page:

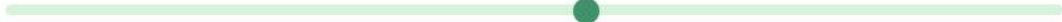


Input Section:


Soil & Climate Parameters
 Adjust all 7 parameters to match your field conditions

Nitrogen (N)

110 kg/ha



Available nitrogen content · range: 0–200 kg/ha

Phosphorus (P)

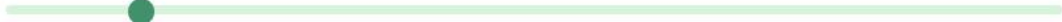
50 kg/ha



Available phosphorus content · range: 0–150 kg/ha

Potassium (K)

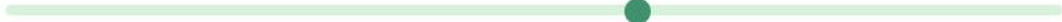
25 kg/ha



Available potassium content · range: 0–210 kg/ha

Soil pH

7.2



Soil acidity / alkalinity · range: 3–10

Temperature

28 °C



Average ambient temperature · range: 5–50 °C

Humidity

60 %

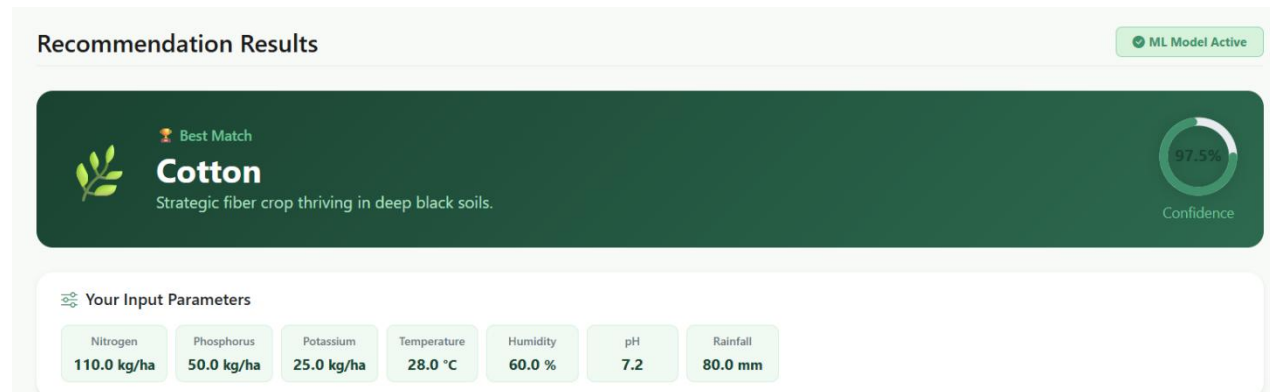


Relative humidity percentage · range: 10–100 %

Rainfall

80 mm

Results Section:



Final Testing Statement

The OptiCrop Smart Agricultural Production Optimization System was successfully tested after deployment. The application accurately processes soil and environmental parameters and provides reliable crop recommendations through a responsive web interface. Functional testing confirmed that the system performs efficiently, generates accurate predictions, and offers a smooth user experience.

Project Conclusion

The OptiCrop Smart Agricultural Production Optimization System is an AI-powered web application that assists farmers in selecting the most suitable crop based on soil nutrients and climatic conditions. By integrating machine learning with a user-friendly web interface, the system improves decision-making, enhances agricultural productivity, and promotes sustainable farming practices. The project demonstrates the practical application of artificial intelligence in agriculture and provides a scalable foundation for future enhancements such as weather API integration, IoT-based soil monitoring, and cloud-based smart farming solutions.